

Análise e Projeto de Algoritmos

Trabalho Prático

Seleção de Postos Centrais de Monitoramento

1 Contexto

Um parque florestal possui diversos postos de monitoramento distribuídos em sua área. Cada posto conta com câmeras, sensores e equipamentos de comunicação. Devido à vegetação, ao relevo e à distância entre os equipamentos, nem todos os postos conseguem comunicar-se diretamente. Quando dois postos possuem comunicação direta, eles podem compartilhar alertas, confirmar ocorrências e transferir informações entre si.

Alguns postos devem operar como centrais de monitoramento, com a função de receber, consolidar e encaminhar as informações dos postos diretamente conectados a ela. Uma central de monitoramento deve:

- receber alertas dos postos vizinhos;
- confirmar ocorrências usando dados de mais de um posto;
- coordenar o acionamento da equipe de segurança;
- armazenar e transmitir registros para a administração do parque;
- assumir o acompanhamento de uma ocorrência que passa de uma área monitorada para outra.

Postos que não são centrais, chamados de postos comuns, observam sua própria região, mas não precisam executar toda a parte de processamento, comunicação externa e coordenação do monitoramento. Eles enviam seus alertas a uma central de monitoramento com a qual possuem comunicação direta.

Como os equipamentos e a manutenção de centrais de monitoramento têm custo elevado, deseja-se escolher um subconjunto dos postos de monitoramento para serem centrais. O objetivo é ter a menor quantidade possível de centrais, garantindo que cada posto satisfaça pelo menos uma das seguintes condições:

- seja escolhido como uma central de monitoramento; ou
- possua comunicação direta com pelo menos uma central de monitoramento.

2 Representação do problema

A rede de monitoramento será representada por um grafo simples e não direcionado $G = (V, E)$, em que:

- cada vértice de V representa um posto de monitoramento;
- uma aresta $\{u, v\} \in E$ indica que os postos u e v conseguem comunicar-se diretamente.

Uma solução é um subconjunto $D \subseteq V$ tal que, para todo vértice $v \in V$, pelo menos uma das seguintes condições seja satisfeita:

- $v \in D$ ou
- $\exists u \in D$ tal que $\{u, v\} \in E$.

O valor de uma solução é dado pelo tamanho do conjunto D , denotado por $|D|$. O objetivo é determinar uma solução de menor cardinalidade possível, ou seja, $|D|$ mínimo possível.

3 Parte A – Parque geral

Nesta parte, a disposição dos postos é arbitrária.

O grupo deverá desenvolver e implementar um **algoritmo exato** para encontrar uma solução de cardinalidade mínima.

Um algoritmo exato deve:

1. produzir uma solução válida;
2. garantir que nenhuma solução menor existe;
3. terminar para toda entrada válida, ainda que o tempo de execução possa ser elevado.

Não será indicada uma técnica específica. O grupo deverá decidir como representar e examinar as possíveis soluções.

3.1 Entrada

O programa deverá aceitar o formato DIMACS:

```
c comentario
p edge n m
e u1 v1
e u2 v2
...
e um vm
```

Os vértices serão identificados por $1, 2, \dots, n$.

O grafo poderá ser desconexo.

3.2 Saída

O programa deverá apresentar:

- o tamanho da melhor solução encontrada;
- os postos escolhidos;
- se a solução apresentada é comprovadamente ótima;
- o tempo de execução;
- o motivo da interrupção, quando aplicável.

Exemplo:

```
Quantidade minima de centrais: 2
Postos escolhidos: 1 4
Otimalidade comprovada: sim
Tempo de execucao: 0.00231 segundos
```

Caso o limite de tempo seja atingido:

```
Melhor solucao encontrada: 37
Otimalidade comprovada: nao
Tempo de execucao: 600.000 segundos
Motivo da interrupcao: limite de tempo
```

3.3 Testes da Parte A

Todos os grupos deverão utilizar o mesmo conjunto de instâncias fornecido pela professora.

As instâncias incluem:

- grafos completos;

- grafos sem arestas;
- estrelas;
- caminhos;
- ciclos;
- grafos desconexos;
- grafos aleatórios;
- grafos esparsos e aproximadamente regulares;
- uma instância grande para observar os limites práticos da solução exata.

Não se espera que todas as instâncias grandes sejam resolvidas com otimalidade comprovada.

4 Parte B – Parque linear

Considere agora um segundo parque, longo e estreito, com uma única trilha principal.

Cada posto monitora um trecho contínuo da trilha, de uma posição ℓ_i até uma posição r_i .

Dois postos comunicam-se diretamente quando existe interseção da área que monitoram.

O objetivo continua sendo escolher o menor número possível de centrais.

4.1 Atividades da Parte B

O grupo deverá:

1. aplicar o algoritmo da Parte A às instâncias do parque linear;
2. analisar quais propriedades adicionais aparecem devido à disposição dos postos;
3. investigar um algoritmo exato que explore diretamente essa estrutura;
4. comparar o novo algoritmo com o algoritmo geral.

Como orientação, o grupo poderá investigar:

- projeto por indução;
- programação dinâmica;
- estratégia gulosa.

Não é necessário implementar uma solução baseada em cada paradigma. O grupo deverá escolher e justificar a abordagem considerada mais adequada.

4.2 Requisitos teóricos

Para o algoritmo desenvolvido na Parte B, o grupo deverá apresentar:

- pseudocódigo;
- prova de que todos os postos são atendidos;
- prova de que a solução é mínima;
- análise de complexidade.

4.3 Testes da Parte B

Cada instância será fornecida em dois formatos:

- arquivo `.col`, contendo o grafo;
- arquivo `.intervalos.txt`, contendo os extremos dos trechos.

O formato do segundo arquivo será:

posto inicio fim

5 Protocolo de medição

O limite máximo de cada execução será de 600 segundos. O algoritmo deverá ser executado de forma sequencial, utilizando apenas uma thread.

Antes das medições, o grupo deverá:

- encerrar programas desnecessários;
- não executar diferentes instâncias simultaneamente;
- não realizar outras tarefas durante a execução;
- utilizar a mesma máquina e as mesmas opções de compilação nos testes comparados;
- manter o computador conectado à energia elétrica;
- evitar modos de economia de energia.

Não será exigida a limpeza manual dos caches do processador ou do sistema operacional. Cada instância concluída deverá possuir uma execução de aquecimento, não registrada, seguida de pelo menos três execuções oficiais. O tempo deverá ser medido com relógio monotônico de alta resolução, por exemplo:

- `std::chrono::steady_clock`, em C++;
- `System.nanoTime()`, em Java;
- `time.perf_counter()`, em Python.

O valor principal apresentado deverá ser a mediana das execuções. Instâncias que atinjam 600 segundos não precisam ser repetidas.

6 Relatório

O relatório deverá conter:

1. modelagem do problema;
2. descrição e pseudocódigo do algoritmo da Parte A;
3. prova de correção da Parte A;
4. análise de tempo e espaço da Parte A;
5. resultados experimentais da Parte A;
6. descrição da estrutura do parque linear;
7. algoritmo proposto para a Parte B;
8. prova de correção e de otimalidade da Parte B;
9. análise de complexidade da Parte B;
10. comparação experimental entre os algoritmos;
11. dificuldades, erros e decisões tomadas durante o desenvolvimento.

O relatório poderá ter entre 6 e 10 páginas, sem contar capa, referências e anexos.

7 Arquivos a entregar

A entrega deverá conter:

1. código-fonte;
2. instruções de compilação e execução;
3. relatório em PDF;
4. arquivo com os resultados brutos;
5. identificação dos integrantes.

Não será permitido utilizar bibliotecas que resolvam diretamente o problema de otimização.

8 Critérios de avaliação

Critério	Peso
Correção e exatidão do algoritmo geral	25%
Prova e análise de complexidade da Parte A	15%
Análise experimental da Parte A	15%
Algoritmo especializado para o parque linear	20%
Prova e análise de complexidade da Parte B	15%
Organização, clareza e reprodutibilidade	10%

9 Observações finais

- Toda fonte consultada deverá ser citada.
- O grupo deverá compreender integralmente o código entregue.
- Qualquer integrante poderá ser solicitado a explicar uma parte do algoritmo ou justificar a resposta produzida.
- Encontrar uma solução válida não é suficiente: quando uma solução for declarada ótima, sua otimalidade deverá estar comprovada.